Tube-Align POC — Team Status Report

OR-Cup (Cup Return) LIFF · 17 August 2026

> **In short:** A feature that helps users aim the camera squarely at the cup-return slot before shooting, so the system reads the glass tube successfully more often.
>
> It runs **on-device** (detection happens on the phone, nothing sent to the server) using a YOLO-pose model that locates the 4 corners of the slot, then computes whether the aim is correct.
>
> **Status:** labeling complete — **production model is currently training**.

1. What this feature is

In the OR-Cup LINE LIFF app, at **Step 3: Tube Scanning**, the user has to photograph the glass tube (the stack of returned cups) at the cup-return machine so the system can count and verify it.

Tube-Align is a real-time aiming assistant:

- While the camera is open, the on-device model continuously detects the cup-return slot in the frame.
- If the user is **off-center, too far, too close, or tilted**, a guidance message tells them how to move.
- Once the aim is **good (good pose)**, the system allows the shot or captures automatically.

Goal: cut down on unreadable photos (errors) at the source, raising the successful-return rate and reducing user retries.

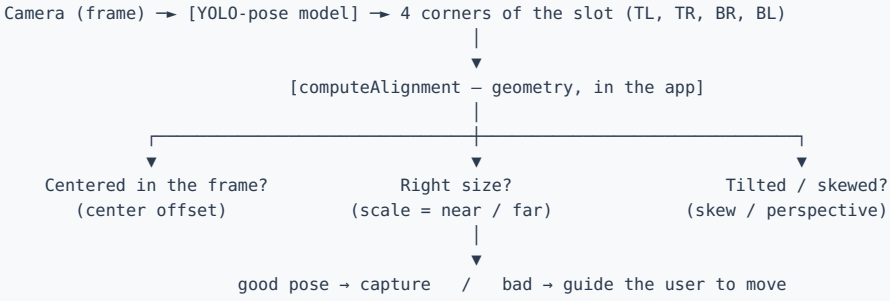
2. Background — why we are doing this

- The original Step 3 was a plain camera where users shot freehand. Many photos were **badly aimed**, so the ML pipeline failed to read the tube.
  - Evidence: the sample images were split into two piles, `success/` (read OK) and `error/` (failed) — the error pile was large enough to be a real problem.
- An **earlier POC** (`tube-align-poc-summary.md`) had already explored the aiming-assistant concept.
- This round of work **builds that POC into something production-ready**: a slot-detection model, a guidance system, and preparation for wiring it into the app.

> **Key insight:** most errors come from poor aiming, which can be fixed at the source (guide before the shot). That is cheaper and simpler than tuning the downstream tube-reading model.

3. How it works (architecture)

There are two parts with **clearly separate responsibilities** — the team needs to be aligned on this point:



- The **model only answers "where is the slot"** (find 4 corners). It has no concept of good or bad.
- The **app decides whether the aim is correct**, from the geometry of those 4 corners compared against a reference (REF).
- Side benefit: to adjust strictness or reword the guidance, we edit the geometry logic in the app — no retraining needed.

Why we labeled error / skewed cases too, not just successes: so the model can still find the corners *even in badly aimed frames*. That is what lets the app know "you are tilted" or "you are too far" and give guidance. Had we trained on successes only, the model would fail to find corners on skewed frames — and then no guidance would be possible.

4. Tech stack

Component	Technology
Model	YOLO11n-pose (4-corner keypoints), Ultralytics
Export	ONNX opset 12 (simplified)
App runtime	onnxruntime-web (WebGPU, with WASM fallback)
Frontend	Next.js 13 (Pages Router, JS), react-webcam, Container / Controller / View
Training	Python + Ultralytics + Albumentations (offline augmentation)
Labeling	CVAT (4-point skeleton label)

Model I/O (for the frontend)

- **input:** `images`, shape `[1,3,640,640]` (NCHW, RGB, /255, letterbox pad 114)
- **output:** `output0`, shape `[1,17,8400]` (17 = 4 bbox + 1 cls + 12 keypoint [x, y, v]) — **NMS is not baked in**, the frontend must do it.
- Note: during live detection the app downsizes via canvas (PROC_W ≈ 320) for speed and fps.

5. Tracks A / B / C — overview

Track	Work	Status
A — Label	Human labeling of the slot (4 corners) on real photos	DONE
B — Train	Build dataset + augment + train → ONNX	IN PROGRESS
C — Integrate FE	Wire the model into the app (onnxruntime-web)	PENDING

6. What has been completed

(a) Frontend mock feature — merged

- The full aiming-assistant flow is implemented in Step 3: detect → compute alignment → on-screen guidance (currently driven by `MockDetector` so the UX could be tested before the real model existed).
- **Merged into the `dev` branch** of `vekin-or-cup-liff`, behind the **feature flag** `NEXT_PUBLIC_ENABLE_TUBE_ALIGN` (default off → the old camera is used, production is unaffected).
- A dev test page exists at `/dev/align`.
- Files: `modules/ReturningCup/alignment/` (computeAlignment, stableCounter, guidance, useDetectionLoop, detector/).

(b) ML pilot (v1 → v4) — proving the concept

- We added labels in rounds to find the real bottleneck: 10 → 37 → 75 → 109 images.
- Pose mAP50 moved: 0 → 0.67 → 0.90 → **0.925**.
- **Conclusion:** the bottleneck is label quantity and quality, not the model — so investing seriously in labeling was worth it.
- v4's limitation: trained on successes only with a small dataset, so it is accurate within a narrow range and does not yet cover real-world cases.
- v4 was then reused as a **pre-label engine** for the production labeling round (people correct labels rather than draw from scratch).

(c) Production labeling (Track A) — complete

- **Curated 600 images** balanced across brightness (dark → bright): 360 success + 240 error, via `curate_prod.py`.
- **Pre-labeled 590/600** with v4, via `prelabel.py` (annotators only drag the 4 corners into place — far faster than labeling from scratch).
- Imported into **CVAT** as the skeleton label `slot` (4 points: TL, TR, BR, BL) through a packaged zip, via `build_cvat_zip.py`.
- The team finished labeling and exported in **"Ultralytics YOLO Pose 1.0"** format: `tube-slot-qa3-20260817.zip`.
- **Totals:** 596 images, **583 labels**
  - 13 images skipped (slot not visible → used as background negatives)
  - 12 images use the `outside` flag (close-up shots where the top corners fall outside the frame)

(d) Label QA — passed

- Wrote `qa_labels.py`: checks for swapped corners, self-intersection (bowtie), out-of-bounds points, winding order, and overlapping corners; compares against v4 (to see how many images were actually corrected); and renders montage grids for a visual overview.
- **HARD errors = 1** (a single out-of-bounds corner at y = −0.013, from dragging a point slightly past the edge) → **auto-fixed by clamping into [0,1]** during dataset build.
- **No swapped corners and no bowties at all** → label quality is good.

(e) Dataset build + augmentation — complete

- `build_from_cvat.py`: clamps out-of-bounds points, recomputes bounding boxes, and does an **80/20 stratified split** (success/error spread across both train and val).
  - train: 467 labeled + 13 background · val: 116 labeled

- `augment.py 3` — Albumentations offline: lighting, glare, shadow, noise, blur, plus mild geometric transforms (**no flips**, since flipping would swap TL ↔ TR). Keypoints are transformed together with the image, and `remove_invisible=False` keeps outside/occluded keypoints.
  - **train: 1,881 (1,868 labeled + 13 background) · val: 116**

7. What remains

After training finishes (closing out Track B)

- [ ] Check **Pose mAP** against v4 (0.925) — see whether coverage of real cases (skewed, far, close) improved.
- [ ] Verify **ONNX I/O** in Netron (input/output shapes per section 4).
- [ ] Test inference on held-out test images the model has never seen.

Track C — Frontend integration (not started)

- [ ] Add `onnxruntime-web` (WebGPU with WASM fallback).
- [ ] Write `OnnxCornerDetector.js` (port of `infer_onnx.py`: letterbox → run → decode 4 corners → NMS). **C0: the skeleton can be written now, while training runs.**
- [ ] Swap `MockDetector` for `OnnxCornerDetector`.
- [ ] Test `/dev/align` on real phones and tune REF (the alignment reference values) to match the real camera and shooting distance.
- [ ] Enable the feature flag for a small user group before wider rollout.

8. Limitations and risks (stated plainly)

> - **mAP is an optimistic number.** The val set is small and comes from the same labeling batch, so real-world numbers may be lower. Real-device testing is required.
> - **No end-to-end testing on real hardware yet.** Frame rate, latency, and real-site lighting (backlight, reflections) are all still unknown.
> - **REF values must be tuned on site.** Real distances and camera angles differ from the training photos, so the good/bad thresholds will need adjusting.
> - **Multiple machine models and locations.** If a slot looks different from the dataset, the model may miss it — additional labeling per machine model may be needed.
> - **An on-device model means size and speed constraints.** yolo11n is already the smallest; if accuracy is insufficient we may have to move to yolo11s, which is slower.